ML Regression Challenge — Spring 2026

Predicting a number (target) from 15 unnamed features (x0–x14)

tl;dr

What I built

A simple linear model:
Standardize the features, then
Ridge Regression.

My result

Score on Kaggle: 0.0496
This is above the competition
Baseline (0.046).

What I learned

A few rare, extreme target values
control the score. The simplest
model handled them most safely.

Author: Shuo Li Tools: Python, scikit-learn (Ridge Regression)

1 EDA — Exploratory Data Analysis

Looking at the data before building any model.

2,500 rows to train, 2,500 to predict

15 features named x0–x14 (we are not told what they mean).

About 1% of values are missing

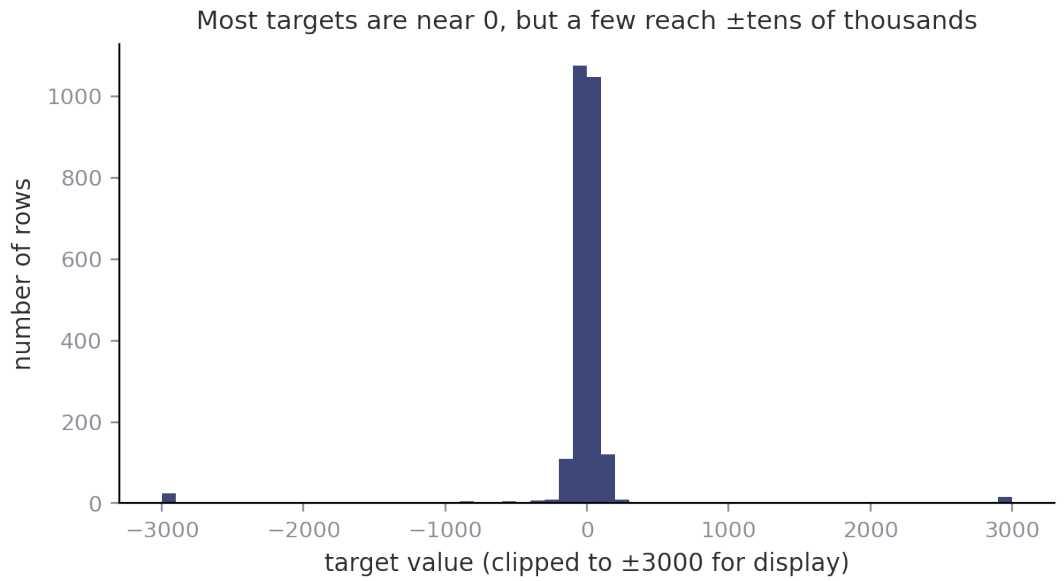
I fill the missing values with each feature's median.

Most target values are small (near 0)

but a few are huge — from about −41,000 to +69,000.

A few points dominate everything

The 10 largest points alone make up ~88% of the total variation, and the score (R^2) mostly depends on them.



Because a handful of extreme values are so large, getting them right (or wrong) decides almost the whole score.

2 Methods I Tried

I started simple, then tried adding complexity to get a higher score.

Method	Why I tried it	What happened	
Linear Regression	The most basic model — a straight-line fit.	Works, but a plain fit is easily thrown off by the extreme values.	tried
Polynomial features	Add squared / combined features to capture curves (Polynomial Regression in class).	More complex, but the score did NOT improve — it became unstable.	tried
Ridge Regression	Linear Regression + regularization to keep the model from over-reacting.	Best and most stable result. This is my final choice.	FINAL

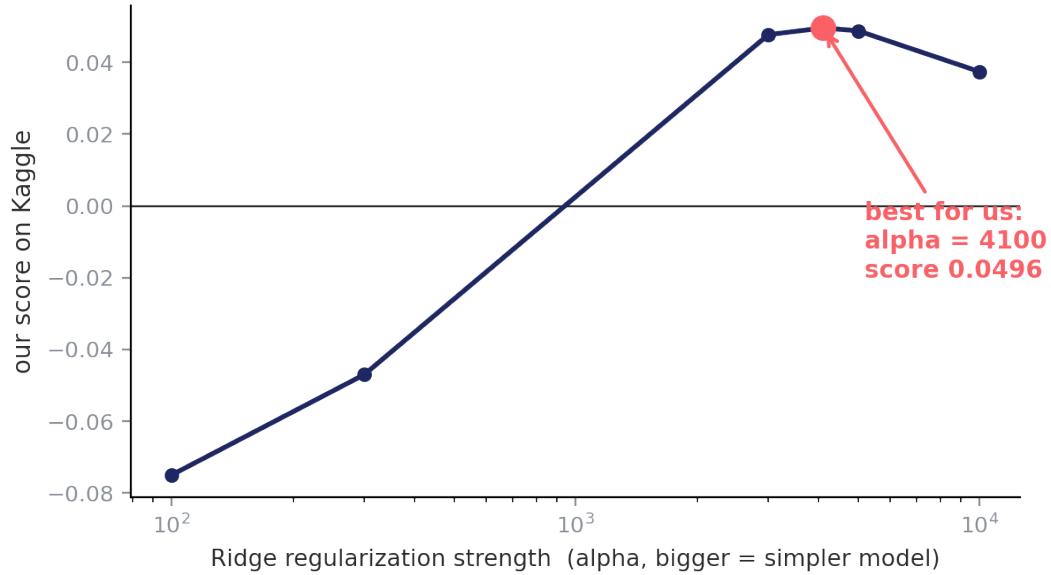
Main takeaway: the more complex methods did not help here. The simple Ridge model was the most reliable — this is the idea of Occam's Razor — prefer the simpler explanation.

3 Why Simple Worked, and How I Tuned It

The problem with complex models. They tried hard to predict the rare extreme values, but those values are basically unpredictable. One bad guess on an extreme point ruins the score (R^2).

Why Ridge is safer. Ridge has a setting called alpha that controls how cautious the model is. A larger alpha makes the predictions gentler, so the model never makes a wild guess that blows up the score.

How I picked alpha. I used cross-validation and the Kaggle score to find the best value. Alpha = 4100 gave my best score.



Reading the chart: too small an alpha → the model over-reacts and the score is negative; too large → it predicts almost the same number for everyone. The best balance is in the middle.

4 Final Model & Code

1

Fill missing values

with each feature's median

2

Standardize features

StandardScaler (put all features on the same scale)

3

Ridge Regression

alpha = 4100

My Kaggle score

0.0496 above the Baseline (0.046)

Code (on GitHub):

github.com/ls1991lsok/ml-hw-spring-2026-python/blob/main/ML_Regression_2026_Code_Shuo_Li.py

The whole solution is about 15 lines of scikit-learn.

Three short steps: fill missing values, put features on the same scale, then fit Ridge Regression.

Conclusion

1. **Look at the data first**

   I saw that a few extreme target values control the score, which changed how I thought about the problem.

2. **Simple beat complex**

   Adding complexity (polynomial features) did not help. The basic Ridge model was the most reliable.

3. **Tune carefully**

   Using cross-validation and the Kaggle score, I chose the Ridge setting (alpha = 4100) that gave my best result.

4. **Result**

   A clean, simple solution that scores 0.0496, above the Baseline.

Thank you!